

從憲法到執行：MAGP 如何重新定義 AI Agent 的工程治理邊界

一份關於 Meta Agent Governance Platform 的深度技術解析

2026 年 6 月

從憲法到執行：MAGP 如何重新定義 AI Agent 的工程治理邊界

一份關於 Meta Agent Governance Platform 的深度技術解析

引言——當 AI Agent 說話，誰來負責？

當一個 AI Agent 說出一句話，這句話的每一個字，都有它的來源嗎？這個問題，在今天大多數的 AI Agent 系統中，答案是：沒有人知道。Agent 的輸出來自訓練資料、RAG 檢索、Prompt 中的假設，或者三者的某種混合——但沒有任何工程機制能夠強制要求 Agent 標示每一個知識主張的來源，也沒有任何正式的規格定義 Agent 能做什麼、不能做什麼。

這是當前 AI Agent 開發的三個根本困境。第一，**知識來源不透明**：Agent 說了什麼，沒有人知道為什麼。第二，**治理規則不正式**：「不要說有害的話」這樣的 Prompt 指令，不是工程規格，而是自然語言的期望——它沒有版本、沒有層級、沒有違規回應機制、沒有可追溯的執行記錄。第三，**架構邊界不清晰**：RAG 系統、Workflow Automation、Prompt 管理器、Agent Executor，這些技術各自解決一個問題，但沒有一個上位框架定義它們之間的關係、邊界與治理責任。

MAGP (Meta Agent Governance Platform) 正是在這個背景下誕生的。它不是另一個 Agent 框架，不是另一個 Prompt 管理器，而是一套試圖從根本上解決「AI Agent 可治理性」問題的 Meta-level 工程規格平台。

「MAGP 是一套用於建構、治理、生成、執行與追蹤 AI Agent 的 Meta-level 平台。它的核心承諾是：從最高治理原則到最小知識單位，每一個環節都有明確的上位依據、明確的輸入輸出規格、明確的治理約束。」

一、為什麼需要 Meta-level 的治理平台？

1.1 傳統 AI Agent 開發的根本缺陷

傳統的 AI Agent 開發依賴 Prompt 工程——用自然語言描述期望行為，希望模型能夠理解並遵守。這種方式在小規模、低風險的場景中或許可行，但在醫療、金融、工業控制等高風險場景中，它的缺陷是根本性的。

知識來源不透明的問題，在高風險場景中意味著法律責任的不可歸因。當一個醫療 AI Agent 給出錯誤的診斷建議，沒有人能夠追溯這個建議來自哪一份醫學文獻的哪一個段落。治理規則不正式的問題，意味著 Agent 的行為邊界是模糊的——今天的 Prompt 說「不要給出具體的藥物劑量建議」，明天換一個工程師修改 Prompt，這條規則可能就消失了，沒有版本記錄，沒有審計軌跡。架構邊界不清晰的問題，在大規模部署時會演變為系統性風險。當一個企業同時運行數十個 AI Agent，每個 Agent 使用不同的 RAG 系統、不同的 Workflow 引擎、不同的 Prompt 管理策略，沒有任何機制確保它們遵守相同的治理原則。

1.2 Meta-level 的工程意義

MAGP 提出的解決方案是 Meta-level 的治理規格。Meta-level 不是「更高層的抽象」，而是「**定義規格應長什麼樣子的規格**」。MAGP 的 Meta-level 定義不針對任何特定 Domain，而是定義所有 Domain 共用的上位規格骨架。

用一個比喻說明：如果 Domain Blueprint 是一棟建築的設計圖，那麼 MAGP 的 Meta-level 規格就是「**建築設計圖必須包含哪些元素、必須符合哪些規範**」的建築法規。建築法規不設計任何一棟建築，但所有建築都必須符合它。MAGP 不定義任何一個 Domain 的具體 Agent 行為，但所有 Domain 的 Agent 都必須在 MAGP 的規格框架下建構。

這個設計的工程意義在於：它將「治理」從 Prompt 工程的範疇，提升到正式工程規格的範疇。

Constitutional Rule 有版本、有層級、有違規回應機制、有可追溯的執行記錄。**Knowledge Item** 有來源標記、有不確定性標記、有精確的粒度定義。**Agent OS** 有九大核心組件、有完整的執行生命週期狀態機、有強制執行的 AuditLog 機制。

二、六大核心層：MAGP 的架構骨幹

MAGP 的正式工程概念架構分為六大核心層，這六層並非平行模組，而是由上而下形成治理與執行的完整體系。每一層回答一個不同的核心問題，共同構成從「最高原則」到「執行期調度」的完整工程路徑。

2.1 Meta Constitution Layer：系統憲法

Meta Constitution Layer 是 MAGP 的最高治理層，在六大核心層中位於最頂端。它定義整個 MAGP 系統不可違反的最高原則，類似一套 Agent 系統的憲法。**Constitutional Rule** 具備五個核心特性：對所有 Domain 均有效；不可被任何下層 Rule 覆蓋；在 Agent OS 執行期由 Governance Checker 強制驗證；違反即觸發執行中止；所有 Constitutional Rule 均繼承自 GovernanceObject Abstract Class，具備可識別性、可版本化、可追溯性。

典型的 Constitutional Rule 包括：「Agent 必須基於可信賴來源回答」、「Agent 無法確認事實時必須標示不確定性 (uncertainty_flag)」、「Agent 執行外部動作前必須檢查授權」、「Agent 的輸出必須可追溯」、「Agent 不得在未授權情況下執行高風險任務」。這些原則不是 Prompt 中的建議，而是在 Agent OS 執行期由 Governance Checker 強制驗證的工程約束。任何 Agent 行為若與 Constitutional Rule 衝突，Agent OS 必須立即中止執行並記錄違規事件。

對所有 Domain 均有效: Constitutional Rule 不針對特定 Domain，跨所有業務場景強制執行，無例外。

不可被下層 Rule 覆蓋: Domain Rule、Item Rule、Execution Rule 均不得與 Constitutional Rule 衝突，違反即中止。

Governance Checker 強制驗證: Agent OS 執行期的 Governance Checker 組件負責即時驗證每個執行節點是否符合憲法原則。

違規觸發執行中止: 違規不是警告，而是立即中止執行並記錄違規事件至 AuditLog，形成可追溯的治理證據。

2.2 Meta Ontology Layer 與 Meta Relation Layer：語意基礎

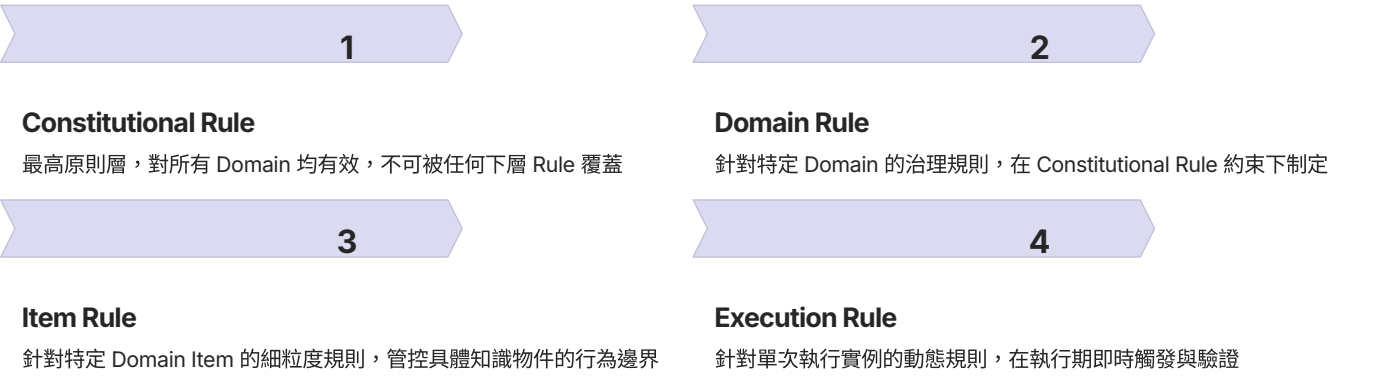
Meta Ontology Layer 定義 MAGP 系統中所有合法的存在類型，回答的核心問題是「系統中有哪些東西？」。它定義了十大 Entity Type：Domain、Agent、Prompt、Workflow、Task、Tool、Memory、KnowledgeSource、ExecutionRecord、HumanApproval。這十個 Entity Type 均繼承自 Entity Abstract Class，共享 entity_id、entity_type、version、status、created_at、updated_by 等基本欄位。

Meta Relation Layer 定義 Entity 與 Entity 之間的合法關係結構，回答的核心問題是「這些存在物之間如何連接？」。它定義了十三種核心 Relation，其中最關鍵的兩種是 **Kind-of**（繼承關係）與 **Part-of**（組成關係）。混淆這兩種 Relation 會導致架構設計的根本性錯誤：Kind-of 表示「A 是 B 的一種」（例如 BloodGlucoseAgent 是 HealthAgent 的一種），Part-of 表示「A 是 B 的一部分」（例如 DiagnosisTask 是 FaultDiagnosisWorkflow 的一部分）。這兩種 Relation 在語意上截然不同，不可混用。

2.3 Meta Governance Layer：四層 Rule 體系

Meta Governance Layer 定義所有治理規則、限制條件、授權條件、風險等級與審查機制。它的四層 Rule 體系形成由上而下的治理層級：**Constitutional Rule**（最高原則，對所有 Domain 均有效）→ **Domain Rule**（針對特定 Domain 的治理規則）→ **Item Rule**（針對特定 Domain Item 的細粒度規則）→ **Execution Rule**（針對單次執行實例的動態規則）。下層 Rule 不得違反上層 Rule，這是 MAGP 治理體系的核心約束。

Rule 不是靜態文件，而是在 Agent OS 執行期間動態觸發的治理機制。每條 Rule 帶有明確的 constraint_target（約束對象）、condition（觸發條件）與 action_on_violation（違規回應）。GovernanceObject 作為所有 Rule 的父類，確保每條 Rule 都具備 rule_id、rule_type、version、scope、condition、action_on_violation 等完整欄位。



2.4 Meta Process Layer 與 Meta Runtime Layer：執行引擎

Meta Process Layer 定義如何將人類意圖轉換為可執行的 Agent 任務，管轄四個 ProcessObject 子類：**Prompt**（結構化的 Agent 執行指令，包含 Intent、Task、Context、Input、Constraint、Source Requirement、Output Requirement 等欄位）、**Workflow**（多步驟任務的執行流程定義）、**Task**（Workflow 中的最小執行單位）、**ExecutionPlan**（由 Intent Parser 生成的完整執行計劃）。這四個子類共同構成從 Human Intent 到 Agent Output 的完整執行鏈。

Meta Runtime Layer 定義執行期間如何調度 Agent、Rule、Workflow、Memory、Tool 與 Human Approval，管轄三個核心 RuntimeObject：**ExecutionContext**（封裝單次執行的完整上下文，包含 Blueprint 快照、Rule 綁定、Intent 解析結果、Memory 快照）、**AuditLog**（記錄每次執行的完整軌跡，包含每個決策點的 Rule 觸發記錄與 KI 引用記錄）、**HumanApprovalRequest**（管理需要人類授權的執行節點）。Agent OS 是 Meta Runtime Layer Abstract Class 在執行期的具體實作。

三、MAGPObject 繼承樹：統一的語意分類系統

MAGPObject 是 MAGP 所有 Abstract Class 的最高父類，統合六大核心層的輸出為三個繼承樹分支：**Entity**（代表可被識別、描述、引用、治理或操作的基本對象）、**GovernanceObject**（代表對 Agent 行為施加約束的治理物件）、**ProcessObject**（代表定義系統如何執行的流程物件）。

這裡有一個關鍵的架構問題需要澄清：六大核心層是 MAGP 的「職責分工」，回答「誰負責定義什麼」；三個繼承樹分支是 MAGPObject 的「語意分類」，回答「這個物件的本質是什麼」。兩者不同維度的劃分，不是一對一的對應關係。Meta Constitution Layer 和 Meta Governance Layer 都產出 GovernanceObject；Meta Ontology Layer 和 Meta Relation Layer 都產出 Entity；Meta Process Layer 和 Meta Runtime Layer 都產出 ProcessObject。

Entity

代表可被識別、描述、引用、治理或操作的基本對象。來自 Meta Ontology Layer 與 Meta Relation Layer 的產出。

- Domain、Agent、Prompt
- Workflow、Task、Tool
- Memory、KnowledgeSource
- ExecutionRecord、HumanApproval

GovernanceObject

代表對 Agent 行為施加約束的治理物件。來自 Meta Constitution Layer 與 Meta Governance Layer 的產出。

- Constitutional Rule
- Domain Rule
- Item Rule
- Execution Rule

ProcessObject

代表定義系統如何執行的流程物件。來自 Meta Process Layer 與 Meta Runtime Layer 的產出。

- Prompt、Workflow、Task
- ExecutionPlan、ExecutionContext
- AuditLog、HumanApprovalRequest

MAGPObject 的四個共同欄位（object_id、object_type、version、created_at）確保所有輸出均具備可識別性、可版本化、可追溯性。這是 MAGP 整個工程體系的基礎設施——無論是 Constitutional Rule、Entity Template、還是 ExecutionPlan，都可以被唯一識別、版本管理、歷史追溯。

四、知識體系：從來源到最小引用單位

MAGP 對知識的處理方式，是整個平台最具工程創新性的部分。它不是簡單地「把文件丟進向量資料庫」，而是定義了一套從知識來源到最小可引用單位的完整工程規格，確保 Agent 的每一個知識主張都有明確的來源標記和不確定性評估。

4.1 三層粒度：KB → KA → KI

MAGP 對知識的組成採用嚴格的三層粒度結構。**Knowledge Block (KB)** 是知識的最粗粒度單位，對應一份完整的知識文件或文件章節。18 冊 Context Type 各對應一個 KB，但 KB 不可直接被 Agent 引用，必須拆解為 **Knowledge Article (KA)**。**Knowledge Article (KA)** 是知識的中層粒度單位，對應 KB 內的一個獨立主題段落，是 Agent 引用知識的中層單位。一個 KB 可拆解為多個 KA。

Knowledge Item (KI) 是知識的最小可引用單位，對應一個具體事實、規則或數值。每個 KI 必須攜帶兩個關鍵欄位：`source_citation`（來源標記，指向具體的 Trusted Knowledge Source）與 `uncertainty_flag`（不確定性標記，標示該知識項的可信賴程度）。這個設計的工程意義是：「有本有據」不是一個口號，而是一個工程約束。Agent 在執行期引用知識時，必須精確定位到 KI 層級，並附上 `source_citation`。缺少 `source_citation` 的 KI 不合規，Agent OS 的 Rule Enforcer 會拒絕引用。

01	02	03
Knowledge Block (KB) 最粗粒度單位，對應完整知識文件或章節。18 冊 Context Type 各對應一個 KB，不可直接被 Agent 引用。	Knowledge Article (KA) 中層粒度單位，對應 KB 內一個獨立主題段落。一個 KB 可拆解為多個 KA，是 Agent 引用的中層單位。	Knowledge Item (KI) 最小可引用單位，對應一個具體事實、規則或數值，必須攜帶 <code>source_citation</code> 與 <code>uncertainty_flag</code> 。

4.2 六種知識包與因果鏈

六種知識包 (Knowledge Package) 是 Domain Blueprint 定義 Agent 推理路徑的核心機制。六個知識包分別是：**觀察包**（定義 Agent 如何觀察問題現象）、**徵狀包**（定義 Agent 如何識別問題徵狀）、**根因包**（定義 Agent 如何分析問題根因）、**診斷包**（定義 Agent 如何確認診斷結論）、**行動包**（定義 Agent 如何建議具體行動）、**策略包**（定義 Agent 如何規劃長期策略）。

六個知識包形成硬性因果鏈：Agent 必須按照「觀察 → 徵狀識別 → 根因分析 → 診斷確認 → 行動建議 → 策略規劃」的順序推理，不得跳步。這是「不亂說話、不亂做事」的工程實現。一個 Agent 不能在沒有完成觀察包和徵狀包的情況下，直接跳到行動包給出行動建議——這種跳步行為會被 Rule Enforcer 識別為違規。



⚠️ **六個知識包形成硬性因果鏈**，Agent 必須按序推理，不得跳步。任何跳步行為——例如在未完成觀察包與徵狀包的情況下直接進入行動包——將被 Rule Enforcer 識別為違規並中止執行。

4.3 18 種 Context Type：知識完整性的觀察框架

18 種 Context Type 由三種類型的視角組合而成：2（尺度：宏觀視角/微觀視角）× 3（對象：Domain 整體/Domain Item/Item 之間的 Relation）× 3（認知層次：描述性知識/分析性知識/評估性知識）= 18 種組合。這個框架確保 Agent 對任何 Domain Item 都能從完整的多維度視角建構知識，避免知識盲點。

以「血糖值」這個 Domain Item 為例，18 冊知識塊涵蓋了從「血糖值的生理機制（微觀描述）」到「血糖值異常對整體健康的影響（宏觀評估）」的完整知識維度。每一冊知識塊都被拆解為多個 KA，每個 KA 進一步拆解為多個 KI，最終形成一個包含 34 個 KI 的完整知識圖譜，每個 KI 都有明確的 `source_citation` 和 `uncertainty_flag`。

尺度維度 × 2 宏觀視角 / 微觀視角	對象維度 × 3 Domain 整體 / Domain Item / Item 之間的 Relation
認知層次 × 3 描述性知識 / 分析性知識 / 評估性知識	= 18 種 Context Type 確保 Agent 對任何 Domain Item 都能從完整的多維度視角建構知識，避免知識盲點。

五、Instantiation Pipeline：從抽象到具體的工程轉換

Instantiation 是 MAGP 生命週期的第二階段，負責將 Meta-level Abstract Class 投影為 Domain-specific Blueprint。這是整個 MAGP 工程路徑的核心轉換點——在這個階段之前，所有定義都是 Meta-level 的抽象規格；在這個階段之後，所有定義都是 Domain-specific 的具體模板，可以被 Agent Factory 用來生成可執行的 Agent 實例。

Instantiation 不是一個單一的轉換步驟，而是六條平行的投影路徑同時進行。Meta Ontology Layer 的 Entity Abstract Class 被投影為 Domain Entity Template（定義 Domain 中有哪些合法 Entity，例如健康管理 Domain 的 BloodGlucoseEntity）；Meta Relation Layer 的 AbstractRelation 被投影為 Domain Relation Template；Meta Governance Layer 的 GovernanceObject 被投影為 Domain Rule Template；Meta Process Layer 的 ProcessObject 被投影為 Domain Process Template；Meta Runtime Layer 的 RuntimeObject 被投影為 Domain Runtime Configuration；Meta Blueprint 的五個規格維度被投影為 Domain Blueprint 的知識結構（KB→KA→KI × 18 種 Context Type × 六知識包）。

Meta Blueprint 是 Instantiation Pipeline 的合規驗證機制，包含五個規格維度：可信賴知識來源清單的建構規格、三層粒度建構規格、六種知識包因果鏈規格、18 種 Context Type 框架規格、四層 Rule 治理約束規格。Meta Blueprint 不是靜態文件，而是 Agent OS 在執行期主動執行的驗證規格。每當 Agent OS 準備載入一個 Domain Blueprint 時，必須先通過 Meta Blueprint 的五個維度驗證，不通過則拒絕載入。這確保了進入執行期的 Blueprint 都符合 Meta-level 規格要求，從源頭防止不合規的 Agent 被部署。

01	02	03
Meta Ontology → Domain Entity Template	Meta Relation → Domain Relation Template	Meta Governance → Domain Rule Template
定義 Domain 中有哪些合法 Entity，例如 BloodGlucoseEntity、SwitchPortEntity	定義 Entity 之間的合法關係，例如 BloodGlucoseEntity 與 DiagnosisEntity 之間的 Diagnoses-of Relation	定義 Domain 的治理規則，例如「血糖值建議必須引用 WHO 或 ADA 指引」
04	05	06
Meta Process → Domain Process Template	Meta Runtime → Domain Runtime Configuration	Meta Blueprint → 知識結構投影
定義 Domain 的執行流程，例如血糖值診斷的六步推理 Workflow	定義執行期的調度配置，例如 Human Approval Gateway 的觸發條件	定義 KB→KA→KI × 18 種 Context Type × 六知識包的完整知識圖譜結構

六、Agent OS：九大組件的執行期調度引擎

Agent OS 不是傳統作業系統，也不是獨立的軟體框架。它是 Meta Runtime Layer Abstract Class 在執行期的具體實作，負責將六大核心層的抽象定義轉化為可執行的 Agent 行為。Agent OS 的完整架構由九個核心組件構成，分為三個功能群組：治理與驗證群組、調度與執行群組、記錄與追蹤群組。

治理與驗證群組包含三個組件。**Governance Checker** 在每次執行前驗證 Constitutional Rule 的最高原則，違反即中止執行——這是 MAGP 治理體系的第一道防線，不可繞過。**Rule Enforcer** 在執行期動態執行四層 Rule，處理違規回應——它不是靜態過濾器，而是在每個執行節點動態觸發的治理機制。**Blueprint Validator** 在載入 Domain Blueprint 前執行 Meta Blueprint 合規驗證，不通過則拒絕載入。

調度與執行群組包含四個組件。**Intent Parser** 將 Human Intent 解析為結構化的 ExecutionPlan，它的輸出不是直接的 Prompt，而是包含完整執行路徑的 ExecutionPlan。**Workflow Engine** 執行 Workflow 與 Task，管理執行流程的狀態轉換。**Tool Dispatcher** 調度 Tool Call，管理工具授權與執行結果。**Memory Manager** 管理知識圖譜、Entity 狀態與 Memory 存取，它管理的是結構化知識圖譜，不是非結構化的向量資料庫。

記錄與追蹤群組包含兩個組件。**Audit Logger** 寫入 AuditLog 與 ExecutionRecord，確保每次執行均可追溯——AuditLog 是 MAGP 可追溯性承諾的工程實現，不可選，必須強制執行。**Human Approval Gateway** 管理需要人類授權的執行節點——它是治理機制，不是可選的 UX 功能。在高風險操作（例如修改網路設備配置、發出醫療建議）前，Human Approval Gateway 必須介入，等待人類授權後才能繼續執行。

治理與驗證群組

- Governance Checker — Constitutional Rule 最高原則驗證
- Rule Enforcer — 四層 Rule 動態執行
- Blueprint Validator — Meta Blueprint 合規驗證

調度與執行群組

- Intent Parser — Human Intent → ExecutionPlan
- Workflow Engine — Workflow 與 Task 執行
- Tool Dispatcher — Tool Call 調度
- Memory Manager — 知識圖譜管理

記錄與追蹤群組

- Audit Logger — AuditLog 強制寫入
- Human Approval Gateway — 人類授權節點管理

七、一個具體的工程場景：VLAN ID Agent 的完整生命週期

抽象的架構描述，最終需要落地到具體的工程場景才能真正理解。以下以「交換機網路管理 Domain」中「VLAN ID Agent」為例，完整展示 MAGP 的工程路徑——從 Meta-level Class Definition 到一次具體的 Agent 執行輸出。

第一階段：Meta-level Class Definition

系統工程師定義 Meta Ontology Layer 的 Entity Abstract Class，其中包含 KnowledgeSource Entity Type（定義知識來源的基本欄位：source_id、source_type、trust_level、last_verified，code style）；定義 Meta Governance Layer 的 Constitutional Rule，包含「Agent 必須基於可信賴來源回答」（rule_id: CR-001，code style）和「Agent 執行外部動作前必須檢查授權」（rule_id: CR-003，code style）；定義 Meta Process Layer 的 Prompt Abstract Class，包含 source_requirement（code）欄位（強制要求每個 Prompt 指定合法的知識來源範圍）。

第二階段：Instantiation

針對交換機網路管理 Domain，Instantiation Pipeline 的六條投影路徑同時進行，產出 Domain Blueprint。

Domain Entity Template

包含 VLANIDEntity（繼承 Entity Abstract Class，擴展 vlan_id、vlan_range、port_membership、trunk_allowed_list，code style 等特有欄位）。

Domain Rule Template

包含「VLAN ID 配置建議必須引用 IEEE 802.1Q 標準或設備廠商官方文件」（Domain Rule，繼承自 CR-001）、「修改 VLAN 配置前必須取得 NOC 授權」（Domain Rule，繼承自 CR-003）。

Domain Blueprint 知識結構

包含 VLAN ID 的 18 冊知識塊、六個知識包因果鏈、36 個 KI 的完整對應矩陣（每個 KI 帶有 source_citation 指向 IEEE_802.1Q_2018 或 Cisco_VLAN_Config_Guide，code style）。

第三階段：Blueprint Generation Engine

Codex 接收 VLAN ID Blueprint Prompt Set，透過 BGE 七大組件（Prompt Parser、Schema Validator、Codex Reasoning Orchestrator、Knowledge Structure Builder、Rule Binding Engine、Blueprint Assembler、Output Validator），生成完整的 Agent Factory Blueprint，包含 VLAN ID Agent 的 Entity 資料模型、Rule Engine 配置、Workflow 模組與知識圖譜結構。

第四階段：Agent OS 執行

當 NOC 工程師詢問「交換機 SW-Core-01 的 Port 24 無法通過 VLAN 100 的流量，可能原因是什麼？」時，Agent OS 的九大組件協同運作：Intent Parser 解析意圖，生成 ExecutionPlan（包含故障診斷推理路徑：觀察包 → 徵狀包 → 根因包 → 診斷包 → 行動包）；Governance Checker 驗證 Constitutional Rule；Memory Manager 從知識圖譜中檢索 VLAN ID 相關 KI（KI-008：

「Trunk Port 必須在 allowed VLAN list 中明確包含目標 VLAN ID，否則流量將被丟棄」，source_citation: IEEE_802.1Q_2018，uncertainty_flag: low；KI-015：「Access Port 只能屬於單一 VLAN，若 PVID 設定錯誤將導致流量分類失敗」，source_citation: Cisco_VLAN_Config_Guide，uncertainty_flag: low）；Rule Enforcer 確認回應符合 Domain Rule；Human Approval Gateway 在生成「建議修改 Trunk allowed VLAN list」的行動建議前觸發，等待 NOC 授權後才輸出具體配置指令；Audit Logger 寫入完整的 AuditLog。

最終的 Agent 輸出：「根據 IEEE 802.1Q 標準（KI-008，source: IEEE_802.1Q_2018），Trunk Port 必須在 allowed VLAN list 中明確包含目標 VLAN ID。診斷結果：SW-Core-01 Port 24 的 Trunk allowed VLAN list 未包含 VLAN 100，導致流量被丟棄（KI-008，uncertainty_flag: low）。建議行動：將 VLAN 100 加入 Port 24 的 Trunk allowed VLAN list（KI-015，source: Cisco_VLAN_Config_Guide）。[此操作需要 NOC 授權，Human Approval Gateway 已觸發，等待授權中]」

這個輸出的每一個知識主張都有明確的 KI 來源，每一個建議都在 Rule 約束下產生，高風險配置操作在 Human Approval Gateway 介入後才能執行，整個執行過程都有完整的 AuditLog 記錄。這就是 MAGP 的核心工程承諾在實際場景中的體現：有本有據、不亂說話、不亂做事——不是口號，而是工程規格。

八、MAGP 的工程邊界：它是什麼，不是什麼

明確定義 MAGP 的工程邊界，對於避免系統定位混淆至關重要。MAGP 是規格，不是實作——它定義「應該怎麼做」的上位規格，而不是「具體怎麼做」的實作細節，不提供可以直接 import 的程式庫。MAGP 是治理平台，不是執行引擎——Agent OS 是執行引擎，MAGP 是定義 Agent OS 必須遵守的規格與約束的治理平台。MAGP 是 Meta-level，不是 Domain-level——所有定義均在 Meta-level 進行，Domain-level 的具體定義是 Instantiation 的產物，不是 MAGP 本身的內容。

MAGP 可以使用但高於以下技術：RAG 系統（在 MAGP 中只是 Knowledge Source Access Mechanism 的一部分，MAGP 定義知識的可信賴規格，RAG 只是檢索工具）；Workflow Automation（在 MAGP 中只是受治理的流程執行工具）；LLM/Codex（在 MAGP 中只是 BGE 的推理核心，MAGP 定義推理的合法邊界）；Agent Framework（在 MAGP 中只是 Agent OS 的底層執行環境）。

✓ MAGP 是什麼

- Meta-level 治理規格平台
- 六大核心層的上位規格定義者
- Agent OS 的規格制定者
- Domain Blueprint 的合規驗證者
- 跨 Domain 一致性的保障機制

✗ MAGP 不是什麼

- 具體的軟體框架或 SDK
- 可直接 import 的程式庫
- Agent 的執行引擎
- RAG 系統的替代品
- Workflow Automation 工具
- 特定 Domain 的解決方案

九、AI Agent 工程的下一個範式

MAGP 代表的不只是一個技術平台，而是一種 AI Agent 工程的範式轉移。從「Prompt 工程」到「治理工程」：治理不再是 Prompt 中的一句話，而是工程規格中的一個正式定義，有版本、有層級、有違規回應機制、有可追溯的執行記錄。從「黑盒子」到「可追溯系統」：AuditLog 機制確保每次 Agent 執行都留下完整的決策軌跡，滿足法律合規、醫療安全、金融監管等高風險場景的要求。從「單一 Domain」到「跨 Domain 一致性」：Meta-level 設計確保所有 Domain 共享同一套上位規格骨架，是大規模 AI Agent 部署的工程基礎。

Prompt 工程 → 治理工程

治理從自然語言期望升級為正式工程規格，有版本、有層級、有違規回應機制

黑盒子 → 可追溯系統

AuditLog 強制記錄每個 KI 引用與 Rule 觸發，滿足法律合規與監管要求

單一 Domain → 跨 Domain 一致性

Meta-level 規格確保所有 Domain 共享相同的治理原則與知識粒度結構

MAGP 目前處於規格定義階段，後續將發布六份下位規格文件：System Architecture Specification、Software Specification、Domain Blueprint Specification、Agent OS Specification、Blueprint Generation Engine Specification 與 Trustworthy Prompt OS Specification。這六份文件將以本規格書所定義的概念、術語與層次關係為共同上位基礎，確保整個 MAGP 工程體系的概念完整性與一致性。

「MAGP 的核心工程承諾是：從最高治理原則（Meta Constitution）到最小知識單位（Knowledge Item），從抽象 Abstract Class 到具體 Agent 執行，每一個環節都有明確的上位依據、明確的輸入輸出規格、明確的治理約束。這是 MAGP 區別於一般 AI Agent 框架的根本所在——它不是讓 Agent 更聰明，而是讓 Agent 更可信賴。」

❶ 本文基於 MAGP 重要概念規格書（第一章至第十七章）撰寫，所有技術定義均以規格書為準。文中所有術語的正式工程定義，請參閱第十六章「標準工程詞彙對照表」。MAGP 後續將發布六份下位規格文件，以本規格書為共同上位概念基礎。

感謝閱讀

本報告由 MAGP 研究團隊撰寫，探討 AI Agent 工程治理的核心架構與實踐路徑。

Meta Agent Governance Platform © 2026